

# Luồng Calendar: Lịch Không Phải Là Một Collection. Nó Là Một View.



Kiến trúc luồng Calendar được xây dựng trên một nguyên tắc cơ bản: các sự kiện lịch không tồn tại trong một collection riêng biệt. Thay vào đó, chúng là một lớp hiển thị (view layer) được tạo ra một cách linh động từ các task hiện có.



# Triết Lý Thiết Kế: Bốn Nguyên Tắc Cốt Lõi



## Single Source of Truth (Nguồn Chân Lý Duy Nhất)

Tasks là thực thể trung tâm. Mọi thông tin đều bắt nguồn từ đây, loại bỏ sự mâu thuẫn dữ liệu.



## Avoid Duplication (Tránh Trùng Lặp Dữ Liệu)

Không cần lưu trữ cùng một thông tin ở hai nơi khác nhau (cả trong task và trong sự kiện lịch).



## Consistency (Đảm Bảo Tính Nhất Quán)

Mọi cập nhật trên một task (ví dụ: thay đổi tiêu đề, trạng thái) sẽ tự động được phản ánh ngay lập tức trên lịch mà không cần đồng bộ hóa.



## Flexibility (Tăng Cường Tính Linh Hoạt)

Các task có thể tồn tại một cách độc lập mà không cần có `dueDate`. Chỉ những task có `dueDate` mới được hiển thị trên lịch.



# Bản Thiết Kế Hệ Thống: Từ Request đến Dữ Liệu





# Nền Tảng Dữ Liệu: Task Model là Nguồn Chân Lý Duy Nhất

## `Task.js` Schema

```
title: String
dueDate: Date (TRƯỜNG KHÓA)
description: String
priority: Enum ['low', 'medium', 'high']
status: Enum ['todo', 'in-progress', 'done']
assignees: Array<ObjectId>
workspace: ObjectId
```

Các trường được làm nổi bật quyết định sự hiện diện, màu sắc, và bộ lọc của một task khi hiển thị trên lịch.

## Indexing Strategy

Đề cập đến các chỉ mục quan trọng như {dueDate: 1, status: 1} để tối ưu hóa các truy vấn dựa trên thời gian và trạng thái.

## Virtual Property

`autoProgress` được tính toán tự động, thể hiện sự phong phú của model.



# Bộ Não Của Não Của Hệ Thống: Calendar Service

## 1. Logic Nghiệp Vụ & Phân Quyền

- Xử lý logic truy vấn phức tạp (kết hợp task cá nhân và task của workspace).
- Kiểm tra quyền truy cập của người dùng trước khi thực hiện bất kỳ thao tác nào (create, update, delete).

## 2. Chuyển Đổi Dữ Liệu (Transformation)

Đây là nơi 'phép màu' xảy ra. Service biến đổi các đối tượng Task thành định dạng Calendar Event mà front-end có thể sử dụng.

Trước

### Task Document

```
{
  "_id": "60d...a1",
  "title": "Design new dashboard",
  "dueDate": "2025-12-25T09:00:00Z",
  ...
}
```

transformToCalendarEvent()

Sau

### Calendar Event Object

```
{
  "id": "60d...a1",
  "title": "Design new dashboard",
  "start": "2025-12-25T09:00:00Z",
  "end": "2025-12-25T09:00:00Z",
  ...
}
```



# Giao Diện Lập Trình Ứng Dụng (API): Cổng Giao Tiếp Với Hệ Thống

| Phương thức (Method) | Đường dẫn (Endpoint)          | Mô tả (Description)                                                 |
|----------------------|-------------------------------|---------------------------------------------------------------------|
| GET                  | /api/calendar/events          | Lấy danh sách sự kiện với các bộ lọc (ngày, workspace, trạng thái). |
| POST                 | /api/calendar/events          | Tạo một sự kiện mới (thực chất là tạo task có `dueDate`).           |
| PATCH                | /api/calendar/events/:id      | Cập nhật chi tiết một sự kiện.                                      |
| DELETE               | /api/calendar/events/:id      | Xóa một sự kiện.                                                    |
| POST                 | /api/calendar/events/:id/move | <b>** (Hỗ trợ Kéo-Thả) **</b> Cập nhật dueDate của một sự kiện.     |
| GET                  | /api/calendar/stats           | Lấy các số liệu thống kê tổng quan về lịch.                         |

 **Ghi chú:** Mọi endpoint đều yêu cầu xác thực (authenticateToken middleware).



# Luồng Hoạt Động: Lấy Dữ Liệu Lịch (GET /events)





# Luồng Hoạt Động: Tạo Mới Sự Kiện (POST /events)



1. **Client:** Gửi yêu cầu `POST /api/calendar/events` với body `{ title, dueDate, ... }`.



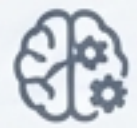
2. **Controller:** Nhận yêu cầu. Gọi `calendarService.createCalendarEvent(eventData, userId)`.



3. **Service: VALIDATION:** Kiểm tra `dueDate` và quyền. Chuẩn bị dữ liệu Task. Gọi `taskRepository.create(taskData)`.



4. **Repository:** Chèn một Task document mới vào MongoDB.



5. **Service:** Nhận lại Task document vừa tạo. Gọi `transformToCalendarEvent()` để chuyển đổi.



6. **Controller:** Nhận lại đối tượng Calendar Event đã chuyển đổi. Gửi phản hồi HTTP **201**.



7. **Client:** Nhận phản hồi thành công và dữ liệu sự kiện mới.

Lưu ý: Một 'sự kiện lịch' không thể được tạo nếu thiếu `dueDate`. Đây là logic nghiệp vụ cốt lõi.



# Các Mẫu Truy Vấn Phổ Biến Cho Giao Diện Người Dùng

## Xem theo tháng (Month View)

```
GET /events?startDate=2025-12-01&endDate=2025-12-31
```

## Xem theo tuần (Week View)

```
GET /events?startDate=2025-12-15&endDate=2025-12-21
```

## Lọc theo Workspace cụ thể

```
GET /events?workspace=workspace_id_123
```

## Chỉ hiển thị Task cá nhân

```
GET /events?workspace=personal
```

## Kết hợp bộ lọc

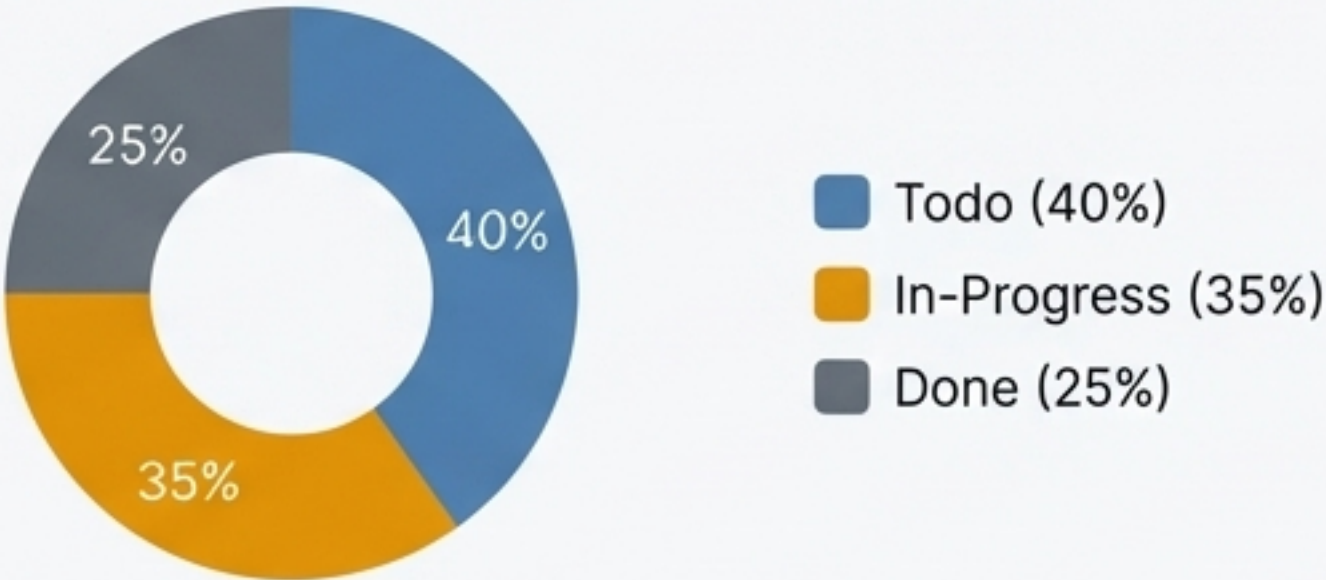
```
GET /events?status=in-progress&priority=high
```



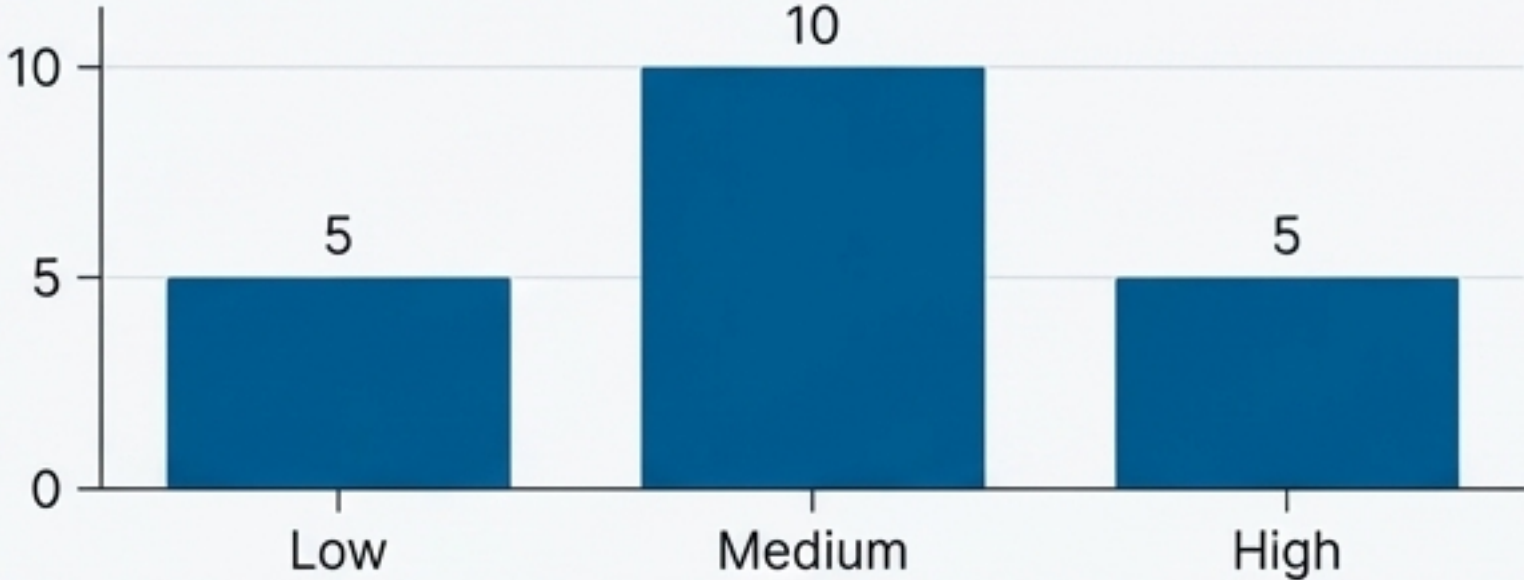
# Không Chỉ Là Lịch: Thống Kê và Phân Tích Dữ Liệu

`GET /api/calendar/stats`

 Phân bố trạng thái Task



 Phân loại theo độ ưu tiên



25%

Tỷ lệ hoàn thành  
(Completion Rate)

3

Số Task quá hạn  
(Overdue Tasks)

10

Số Task sắp tới  
(Upcoming Tasks - 7 ngày)

Tất cả các số liệu này được tính toán trong thời gian thực dựa trên cùng một nguồn dữ liệu Task.



# Tối Ưu Hiệu Năng: Xây Dựng Để Nhanh và Tin Cậy

## Cấp Cơ Sở Dữ Liệu (Database Level)



**Chỉ mục (Indexes):** Sử dụng các chỉ mục kết hợp (`{dueDate: 1, status: 1}`) để tăng tốc các truy vấn lọc theo ngày và trạng thái.



**Tối ưu truy vấn (Query Optimization):** Lấy workspaceIds của người dùng một lần, sau đó sử dụng toán tử `$in` để giảm số lượng truy vấn.



**Population có chọn lọc:** Chỉ populate các trường cần thiết để giảm tải.

## Cấp Ứng Dụng (Application Level)



**Lớp chuyển đổi gọn nhẹ:** Hàm transform không thực hiện thêm truy vấn nào.



**Caching cho Statistics:** Đề xuất sử dụng cache (ví dụ: `Redis`) cho kết quả của `getCalendarStats`.

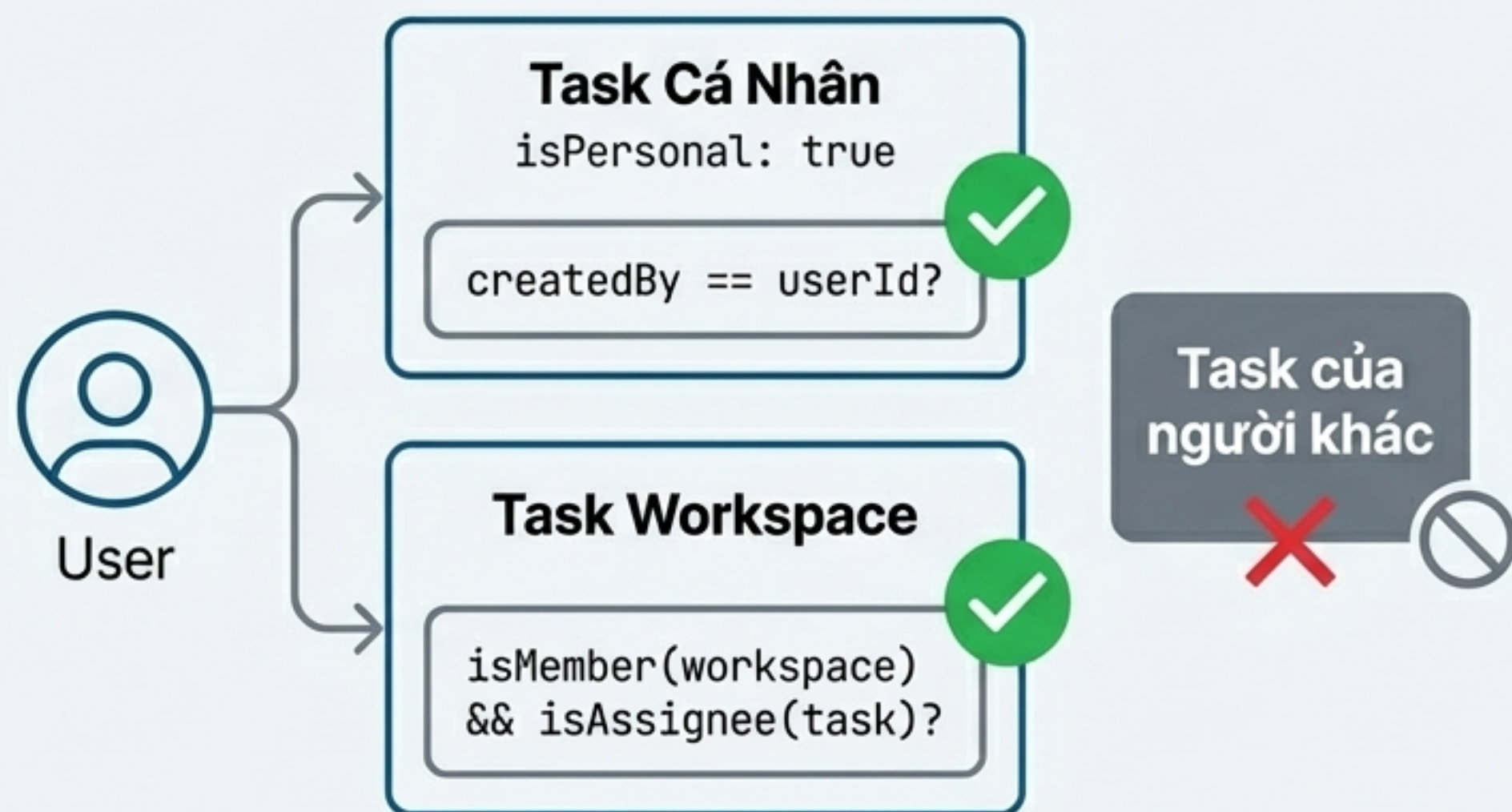


**Giới hạn khoảng thời gian:** Áp dụng giới hạn tối đa cho khoảng thời gian truy vấn (ví dụ: 3 tháng) để ngăn chặn quét toàn bộ collection.



# An Toàn và Bảo Mật: Kiểm Soát Truy Cập Dữ Liệu

## Mô hình phân quyền



## Các quy tắc chính

- Người dùng chỉ có thể truy cập các task mà họ **tạo ra** HOẶC **được gán vào**.
- Đối với các task thuộc workspace, hệ thống luôn kiểm tra tư cách thành viên (**isMember**) của người dùng.
- Đối với các task cá nhân (**isPersonal: true**), hệ thống kiểm tra xem người dùng có phải là **createdBy** hay không.

Thiết kế này đảm bảo rằng người dùng không bao giờ thấy được dữ liệu mà họ không được phép, dù truy cập qua API Task hay API Calendar.



# Xử Lý Lỗi: Các Tình Huống Thường Gặp

| Tình Huống (Scenario)       | Thông Báo Lỗi (Error Message)              | Mã Trạng Thái (Status) |
|-----------------------------|--------------------------------------------|------------------------|
| Tạo sự kiện thiếu ngày      | Calendar events must have a due date       | 400 Bad Request        |
| Truy cập trái phép          | You are not authorized to access this task | 403 Forbidden          |
| Workspace không tồn tại     | Workspace not found                        | 404 Not Found          |
| Định dạng ngày không hợp lệ | Invalid date format                        | 400 Bad Request        |

## Định dạng phản hồi lỗi

```
{
  "success": false,
  "message": "Error description"
}
```



# Lộ Trình Phát Triển và Khả Năng Mở Rộng



## Hạn Chế Hiện Tại (Current Limitations)

- Chưa hỗ trợ phân trang (pagination) cho sự kiện.
- Số liệu thống kê được tính toán theo yêu cầu, chưa có caching.
- Chưa hỗ trợ sự kiện lặp lại (recurring events).



## Đề Xuất Cải Tiến (Recommended Improvements)

- **Phân Trang:** Giới hạn số lượng sự kiện trả về và lazy load khi người dùng cuộn.
- **Caching:** Triển khai Redis cache cho `getCalendarStats`` với TTL (Time-To-Live).
- **Tính Năng Mới:**
  - Hỗ trợ sự kiện lặp lại.
  - Thông báo nhắc nhở sự kiện.
  - Chia sẻ lịch (quyền chỉ đọc).
  - Tích hợp với các dịch vụ lịch bên ngoài.



# Tổng Kết: Calendar là một Lớp Trừu Tượng Tinh Gọn

**'Calendar' không phải là một thực thể dữ liệu, mà là một lăng kính (lens) để nhìn vào dữ liệu Task hiện có.**

- ✓ **Nguồn Chân Lý Duy Nhất:** Loại bỏ hoàn toàn việc đồng bộ hóa dữ liệu.
- ✓ **Hiệu Quả Vận Hành:** Thao tác trên lịch chính là thao tác trên task, đơn giản hóa logic.
- ✓ **Linh Hoạt Tối Đa:** Dễ dàng mở rộng các tính năng liên quan đến thời gian mà không cần thay đổi cấu trúc dữ liệu gốc.

